to associate the user account with that data. Django provides two ways of defining account-associated storage:

- Using a profile ORM object
- Replacing the `User` ORM object with a custom model

A user profile object is a separate `Model` with a special kind of foreign key pointing to the Django `User` account object. The profile object stores your custom data, while the `User` `Model` provides shortcuts for access through the relationship. The special foreign key is called a `OneToOneField`, and it constrains the relationship so that a profile object can only be associated with a single user account object, and vice versa. The `OneToOneField` isn't specific to profile objects; you can use them in any code where you wish to have a foreign key relationship in the database with a single relation restriction.

The second option is the hardcore, I-want-to-customize-everything-option. The user profile approach locks you into using the Django `User` `Model`'s credential mechanism. Replacing the `User` object altogether allows you to replace the credentials, for example, using an email address instead of a username as the login. Django's flexibility around user accounts comes at a complexity cost: in addition to the `User` object, there is also a user manager object. To replace the `User` altogether, you also have to replace the manager object. Figure 6.2 compares the user profile and replacement approaches, showing both the objects and related database changes.

Choosing which customization option to use is important, as changing your project in the future is painful. Django is only capable of using one user account mechanism at a time, and switching from one style to the other usually requires custom migration scripts. You also have a potential data problem: consider using usernames and then switching to email address as a login credential. You need to make sure all of your users have email addresses in the system and that they are unique. This is a design decision you should make as close to the beginning of your project as possible—and that you should stick with.

As the hardcore option is, well, hardcore, this book will stick to the user profile case. If you're interested in more customization than is covered in this chapter, see https://mng.bz/RZZn.

Proxy models

Django allows a third option for customizing the user account if all you want is behavioral changes. An ORM model can be declared a *proxy model* by inheriting from the model being proxied and adding `proxy = True` to the model settings in the `Meta` subclass.

A proxy model object uses the same table as its parent; the attributes and their corresponding columns are the same. Both the proxy and its parent abstract the same rows in the same table. What can change in a proxy is its behavior. For example, you can add a different `ordering` attribute to the `Meta` subclass, and your default sorting of the proxy will be different from its parent.